TrustVault: Secure Document Sharing with an Audit Trail

README | Snovasys Fresher Hiring Challenge 2026

1. Introduction

TrustVault is a web application for sharing documents safely. A user can upload a file, choose who is allowed to open it, and later take that permission back. The system also keeps a record of what happened, including the times when someone tried to open a file they were not allowed to see.

I made this project for the Snovasys hiring challenge. At the start I thought the hard part would be uploading files, but after reading the brief I understood that uploading is easy. The real work is making sure the correct person gets the correct file and that there is proof of who accessed what. Most of my time went into that part.

I used AI tools while building it. How I used them is written in PROMPTS.md. The risks I thought about, and the ones I could not fully solve, are written in THREATS.md. This file explains what the project does and how to run it.

2. The main idea

Think about a college where confidential documents are shared, such as marks, reports or internal notes. If the files sit in a normal shared folder, anyone with the link, or anyone who can guess an ID, may open them. Also, if something goes wrong, nobody can say who looked at the file.

In TrustVault the browser is never trusted. Hiding a button in the React page does not protect anything because a person can send a request by hand. So every request goes to the Express backend, and the backend checks who the user is and whether that user has permission for that exact document before doing anything.

3. User roles

Role	What the role can do
Admin	Can view the audit log and check suspicious activity. Admin does not get free access to every file just because of the role. Admin actions on documents are recorded under separate names in the log.
Owner	Uploads documents, manages them, and gives or removes access for other users.
Viewer	Can read or download only the documents that were shared with them. Changing the ID in the URL does not give access to anything else.

4. Features

- Login with a JWT token. Passwords are stored only as bcrypt hashes.
- Permission checks on the server for reading, downloading, editing and sharing.

- Upload, list, search and download of documents. A user sees only the documents they are allowed to see.
- Sharing with two levels, read and edit. The owner can remove access, and the next request from that user is checked again, so the removal works immediately.
- Audit log for uploads, downloads, permission changes and denied attempts.
- Files are kept in a private folder on the server under random names and are not served as public files.
- Automated tests with Jest and Supertest that try forbidden actions and expect them to fail.

5. Technologies used

Part	Technology	Reason
Frontend	React with Vite	I already knew HTML, CSS and JavaScript, and React makes forms and lists easier.
Backend	Node.js and Express	Same language as the frontend. Middleware is a good place for login and permission checks.
Login	JWT and bcryptjs	JWT works well with a React frontend. bcryptjs hashes the passwords.
Security setup	Helmet and CORS	Helmet adds security headers. CORS limits which website can call the API.
Uploads	Multer	Handles file upload with size limit and type filter.
Database	MySQL	Tables, foreign keys and unique keys fit this data well.
Storage	Private folder on the server	Simple for this project. Its problems are listed in section 16.
Testing	Jest and Supertest	To call the real API and check the status codes.

Installing these packages does not make an application secure by itself. They have to be configured correctly and tested, which is why I wrote the tests and the threat document.

6. How a request is handled

Here is what happens when a viewer presses the download button:

1. The React app sends a request to the Express API with the JWT in the header.
2. The authentication middleware checks the token. If it is missing or wrong, the API answers 401.
3. The backend reads the role of the user from MySQL. It does not accept a role sent by the browser.
4. It then checks whether the user owns the document or has a row in the document_permissions table. The queries are parameterized.

5. If there is no permission, a denied entry is written to the audit log and the API answers 403. The file is not opened.
6. If there is permission, the backend finds the stored file name, sends the file and writes a success entry in the audit log.
7. The frontend shows the file or an access denied message.

The database keeps only the details of the file and the permissions. The file itself is in the private folder. Because of this, a database row alone never gives a file to anybody.

7. Folder structure

This is the structure I follow. The repository is the final reference if something is different.

```
TrustVault/
  client/               React + Vite frontend
    src/components/     reusable parts
    src/pages/          login, dashboard, documents, admin
    src/services/       API calls
  server/               Express backend
    server.js           starts the server
    db.js               MySQL connection
    routes/             API routes
    controllers/        request handling
    middleware/         login and role checks
    tests/              Jest and Supertest tests
    private_uploads/    uploaded files (not in Git)
  README.md  THREATS.md  PROMPTS.md
  .env.example  .gitignore
```

8. Database

The database is called trustvault and has four tables. The details below come from my own database export, so they match what I actually ran.

Table	What it stores	Notes
`users`	Accounts and roles	Email is unique. Passwords are stored as bcrypt hashes. Role is admin, owner or viewer, and the default is viewer.
`documents`	File details	Owner id, name, category, mime type, size and a unique storage key (a random UUID file name).
`document_permissions`	Sharing	Permission is read or edit. It also stores who granted it. The pair (document, user) is unique, so a user has only one permission per document.
`audit_logs`	Activity record	User, document, action, result (success or denied), details in JSON and the time.

A few choices I made on purpose. The audit table uses ON DELETE SET NULL, so if a user or document is deleted the old log rows are not lost. The storage key is random, so nobody can guess the real path of a file. And the unique key on permissions stops duplicate grants.

9. Security measures

Measure	Why it is there
JWT on protected routes	To stop anonymous access
Ownership and permission check on the server	To stop one user opening another user's document by changing the ID
Role taken from the database	So a user cannot claim a higher role
Parameterized queries	To prevent SQL injection
bcryptjs hashing	So stolen database rows do not show real passwords
Upload limits and type filter	To block very big files and unwanted types
Random names, private folder	So files cannot be guessed or opened directly
Helmet and CORS	Security headers and a limited allowed origin
Audit log	To show what happened, including failed attempts
Automated negative tests	To catch mistakes that reopen a hole

10. Permission example and audit log

Suppose User A uploads salary.pdf and shares it with User B as reader. User A can download it. User B can download it. User C is denied. If User B tries to edit it, that is denied too because B has only read access. After User A removes the access, B is denied on the very next request.

My database shows that this is really recorded. The audit table has actions like document_upload, document_read, document_download, permission_grant and permission_revoke, and also separate admin_document_read, admin_document_download and admin_document_delete. Each entry is marked success or denied.

I record the denied attempts on purpose. A successful download only shows who got a file, but many denied attempts show that someone is trying to get in. Passwords, tokens and file contents are never written to the log.

11. Upload rules

- Allowed types are PDF, TXT, JPEG and PNG.
- The maximum size is 5 MB for each file.
- The file is saved with a random UUID name in a private folder.

- Type and extension checks are only a first filter. They do not prove that a file is safe, so scanning is listed under future work.

12. How to run the project

You need Node.js with npm, MySQL Server and Git. I worked on Windows with PowerShell.

Step 1. Clone the repository.

```
git clone <repository-url>
cd TrustVault
```

Step 2. Create a MySQL database named trustvault and create the four tables from the schema or the SQL export. With an export file it can be loaded like this:

```
mysql -u your_mysql_username -p < trustvault-backup.sql
```

The SQL export has emails and password hashes in it, so *.sql is in .gitignore. It should never be pushed to GitHub.

Step 3. Install the backend and create the .env file inside the server folder.

```
cd server
npm install
```

```
PORT=5000
DB_HOST=localhost
DB_PORT=3306
DB_USER=your_mysql_username
DB_PASSWORD=your_mysql_password
DB_NAME=trustvault
JWT_SECRET=use_a_long_random_secret
FRONTEND_URL=http://localhost:5173
```

The names must match what the backend code reads. The .env file must not be committed.

Step 4. Start the backend with npm start. If a dev script exists in package.json, npm run dev also works. The default port is 5000.

Step 5. Start the frontend in another terminal.

```
cd client
npm install
npm run dev
```

Open the address that Vite prints, usually http://localhost:5173, and make sure the frontend is pointing to the backend address.

Step 6. Run the tests from the server folder with npm test. The database must be set up for the tests to work.

13. Using a cloud MySQL database

I also made a second copy of the database export for Aiven, which is a hosted MySQL service. It has the same data but without the CREATE DATABASE and USE lines, because Aiven already gives a database. Hosted databases usually need a secure connection, so the backend can read a CA certificate file (server/ca.pem). That file is in .gitignore. I have not put any deployment link or credentials in this README.

14. API areas and tests

Area	**Purpose**
Authentication	Login, register and get the current user (for example /api/auth/me)
Documents	Upload, list, search, view or edit details, download
Permissions	Grant and revoke read or edit access
Administration	Read and filter the audit log (for example /api/admin/audit-logs), admin only

I did not copy every route here, because it may change. The route files are the full list.

The tests try things a user should not be able to do. They check that a user cannot read or download another user's document, that a request without login is refused on the audit log endpoint, and that a normal user without admin role is refused too. Passing tests only prove the cases that were tested, so I do not say the application is completely secure.

15. Demo steps

1. Log in as an owner and upload a small PDF.
2. Log in as another user and try to open that document. It should be denied.
3. As the owner, give read access to the second user. Now it opens, but editing is still denied.
4. Remove the access and try again. It should be denied right away.
5. Log in as admin and open the audit log to show the successful and denied entries.
6. Run npm test to show the security tests.

16. Limitations

- Files are stored on the local disk of the server. If the folder is deleted or the hosting platform resets its disk, the files are lost but the database rows remain. There is no backup yet.
- Nobody outside the project has audited it. I only tested selected cases.

- The audit log is a normal database table, so a person with database access could change it.
- Upload checking is basic. There is no virus scan or deep file check.
- All screens of the frontend should be tested again on the latest build before saying everything works.
- It is not ready for real confidential data without more work on deployment, monitoring and backups.

17. Future work

- Private cloud storage with backup and recovery.
- File signature check and malware scanning.
- Rate limiting for login, and better password rules.
- Better token handling (expiry, refresh, revoke).
- More tests for uploads, edits and permission changes.
- More filters and pagination in the audit log, and a way to detect tampering.

18. Common problems

Problem	**What to check**
CORS or network error in the browser	Is the backend running? Does FRONTEND_URL in .env match the Vite address and port exactly?
Backend cannot connect to MySQL	Host, port, user, password and database name in .env. Also check that MySQL is running.
Every request gives 401	Token may be missing or expired. Log in again. JWT_SECRET should not have changed.
Upload rejected	File type must be PDF, TXT, JPEG or PNG and under 5 MB.
Download says file not found	The database row exists but the file is missing from the upload folder.
Tests fail on a new computer	Check the database and the environment variables.

19. What I learned

The main thing I learned is that login and authorization are different problems. Making a user log in is the easy part. The difficult part is to ask on every request whether this person is allowed to do this action on this document. Writing tests that try to break the rules also changed how I think, because I started to imagine how someone could misuse each route and not only how a normal user would use it.

I also learned that AI can write code quickly, but it does not take responsibility for it. I still had to read it, run it, break it and fix it.

20. Note

TrustVault is a learning project made for a hiring challenge. It shows some good security practices, but it is not a fully audited or production system and should not be used for real confidential data without more hardening.